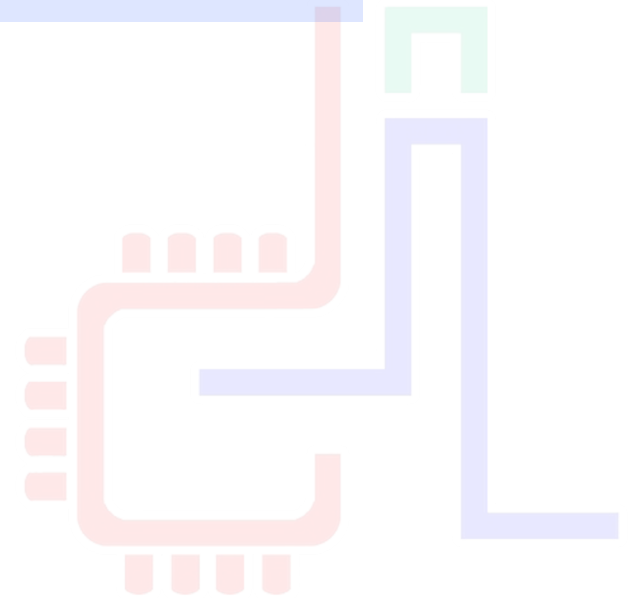


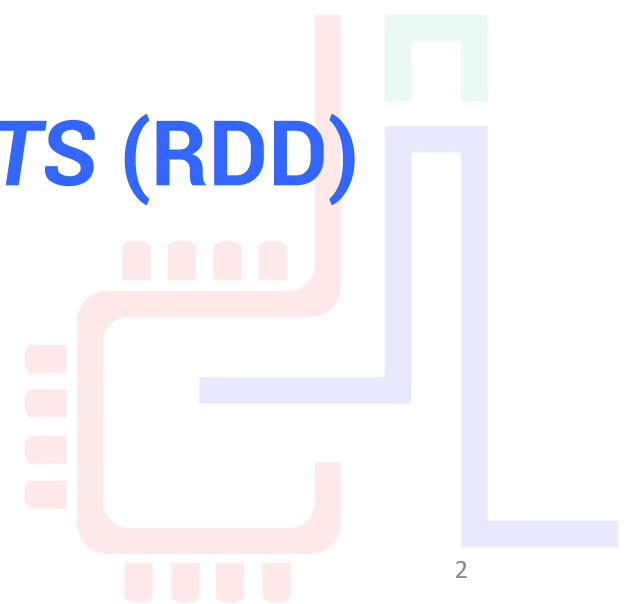
Framework BigDATA

TAREK BEN MENA



RESILIENT DISTRIBUTED DATASETS (RDD)

Partie 2



Resilient Distributed Datasets (RDD)

- C'est le concept central : Un RDD est une collection calculée à partir d'une source de données (MongoDB, un flux, un autre RDD).
 - Un RDD peut être marqué comme **persistant**: il est alors placé en mémoire RAM et conservé par Spark.
 - Spark conserve l'**historique des opérations** qui a permis de constituer un RDD, et la reprise sur panne s'appuie sur cet historique afin de reconstituer le RDD en cas de panne.
 - L'idée est qu'il est plus facile et efficace de **préserver quelques lignes de spécifications** que le jeu de données issu de cette chaîne.
- Un RDD est un "bloc" **non modifiable**. Si nécessaire il peut-être entièrement recalculé.

RDD : Exemple d'un Job

```
wordcount.py > ...
1  import sys
2
3  from pyspark import SparkContext, SparkConf
4
5  if __name__ == "__main__":
6      # create Spark context with Spark configuration
7      conf = SparkConf().setAppName("Word Count - Python")
8      sc = SparkContext(conf=conf)
9
10     # read in text file to RDD
11     lines = sc.textFile("./input.txt")
12
13     # split each line into words
14     words = lines.flatMap(lambda line: line.split(" "))
15
16     # count the occurrence of each word
17     word_with_1 = words.map(lambda word: (word, 1))
18
19     # reduce the occurrence of each word to one value (word_counts is RDD)
20     word_counts = word_with_1.reduceByKey(lambda a,b:a +b)
21
22     #Collect the RDD to a list
23     result = word_counts.collect()
24
25     for (word, count) in result:
26         print(word, count)
27
```

RDD

RDD

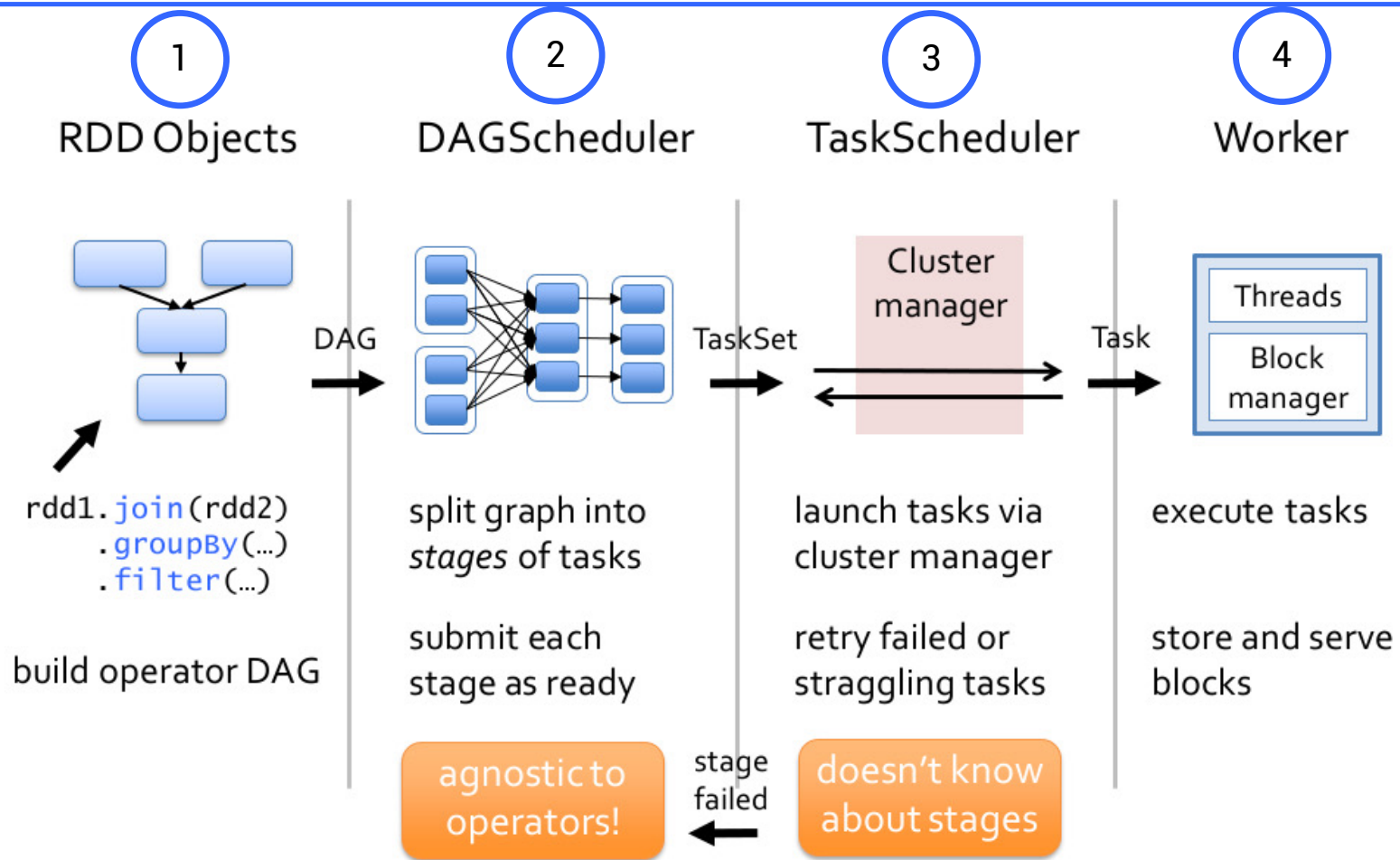
RDD

RDD

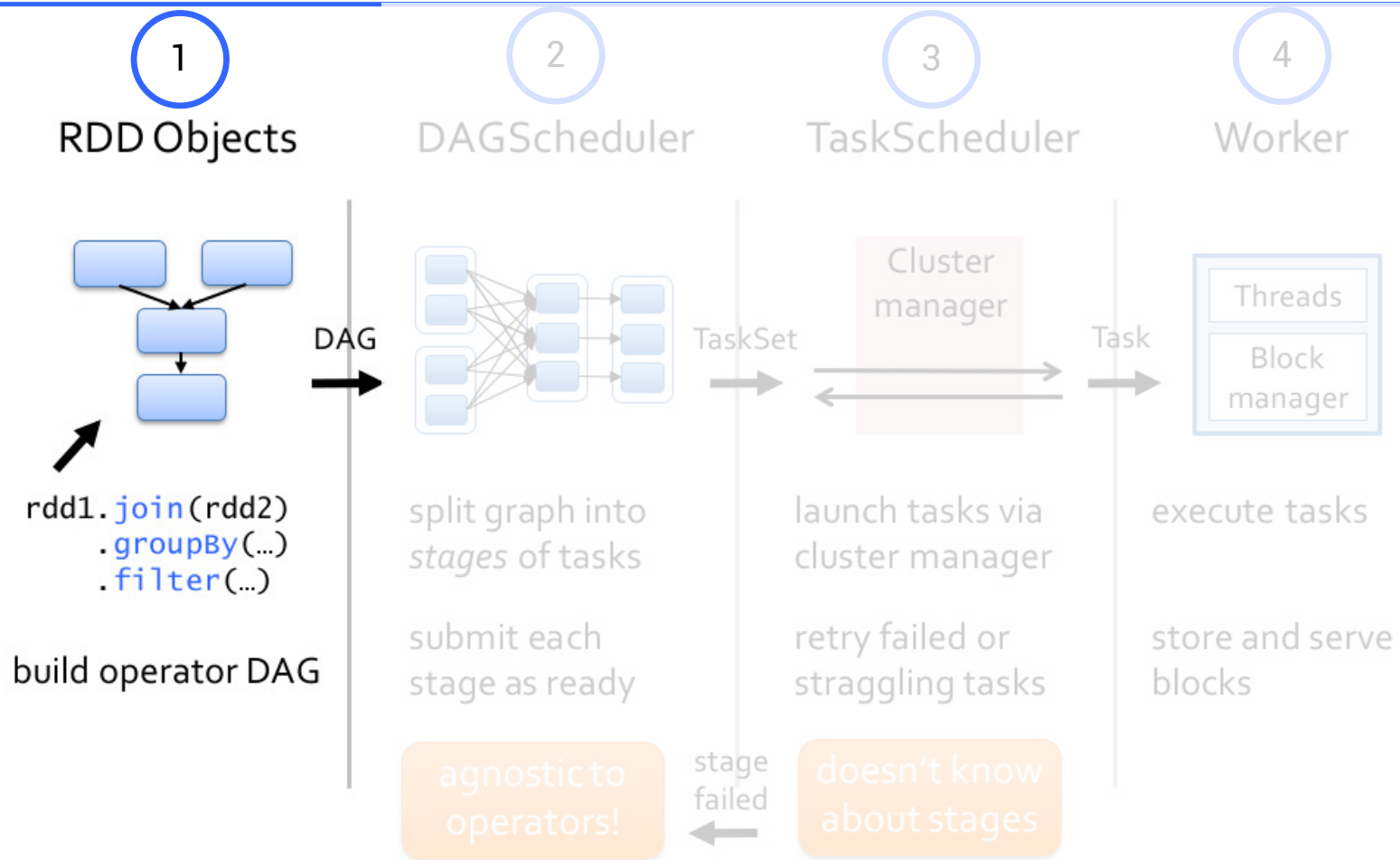
Transformations

Action

Spark Job Scheduling

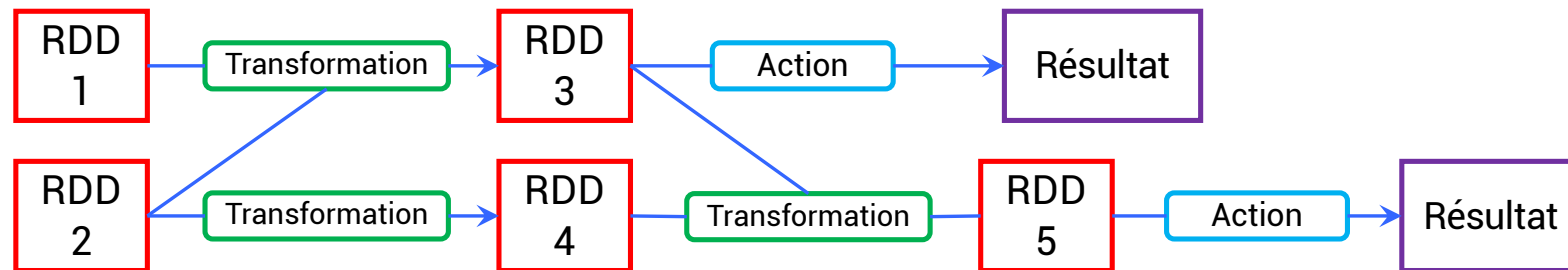


Spark Job Scheduling



DAG : Directed Acyclic Graph

- Dans une application Spark, les **transformations** et les **actions** réalisées sur les **RDD** permettent de construire un **graphe acyclique orienté** (DAG)



- **Tolérance aux pannes**

- Lorsqu'un nœud (RDD, Résultat) devient indisponible, à cause d'une mal-fonction,
- il peut être régénéré à partir de ses nœuds parents.

RDD : Création

- Le `SparkContext` crée un RDD

```
conf = SparkConf().setAppName("Word Count - Python")
sc = SparkContext(conf=conf)
```

- il est possible de lire les données à partir d'une grande variété de sources différentes.
- `sc.textFile(path)` crée un RDD à partir des **lignes** du fichier localisé par son path (peut être un chemin réseau)
- `sc.wholeTextFiles(path)` path : un chemin vers un fichier ou un répertoire contenant plusieurs fichiers.
 - Ensemble de **(clés-valeur)**
 - **clé** : le chemin vers un fichier
 - **valeur** : contenu complet des fichiers.
 - Le(s) fichier(s) ne sont donc pas décomposés en lignes.
- `sc.parallelize(iterable)` utilisé pour transformer des objets Python (ou Scala) en RDD. `parallelize` (utilisé dans le Spark Shell pour déboguer).

RDD : Transformations

- `map(func)`: applique une fonction `func` à chacune des données.
- `filter(func)`: permet d'éliminer certaines données.
 - Ex : dans `wordcount.py`, pour ne sélectionner que les mots commençant par une majuscule :
`words.filter(lambda word: word[0].upper() == word[0]).`
- `flatMap(func)`: tout comme `map`, mais chacune des données d'entrée peut être transformée en plusieurs données de sortie.
- `sample(withReplacement, fraction, seed)`: récolte un échantillon aléatoire des données. Cette méthode est utile pour tester un algorithme sur un petit pourcentage de données. L'argument `seed` permet de réaliser des expériences reproductibles.
 - Ex : `.sample(False, 0.01, 42).`
- `distinct()`: supprime les doublons.
- `groupByKey()`: transforme des clés-valeurs (K, V) en (K, W) où W est un objet itérable.
 - Ex: (K, U), (K, V) et (K, Z) seront transformées en (K, [U, V, Z]).

RDD : Transformations

- `reduceByKey(func)`: applique une fonction de réduction aux valeurs de chaque clé.
 - la fonction de réduction `func` est appelée avec deux arguments.
 - `func` est commutative et associative, c-à-d
 - `func(a, b) = func(b, a)` et `func(func(a, b), c) = func(a, func(b, c))`.
- `sortByKey(ascending)`: pour trier le résultat par clé.
 - Exemple, pour obtenir la liste de tous les mots triés par ordre alphabétique,
`wordcounts = wordcounts.sortByKey()`
- `join(rdd)`: pour réaliser une jointure, ce qui a le même sens que dans les bases de données relationnelles.
 - il existe également des fonctions `leftOuterJoin`, `rightOuterJoin` et `fullOuterJoin`
- `union(rdd)`: pour réaliser une union entre deux collections de RDD

RDD : Actions

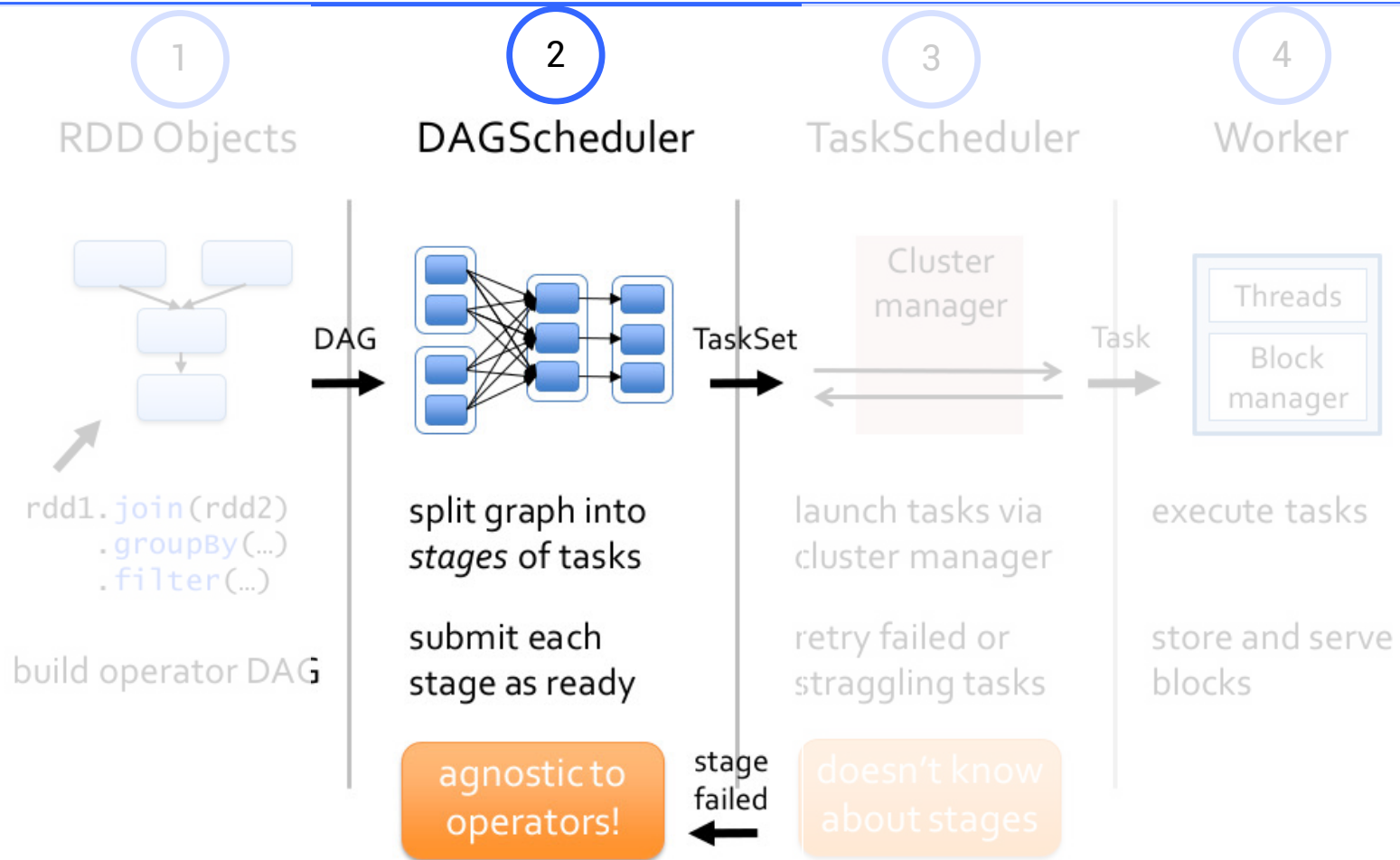
- `reduce(func)`: applique une réduction à l'ensemble des données.
- `collect()`: retourne toutes les données contenues dans le RDD sous la forme de liste.
- `count()`: retourne le nombre de données contenues dans RDD.
- `takeOrdered(n, key_func)`: retourne les `n` premiers éléments du RDD ordonnés selon `key_func`.
 - Exemple, pour obtenir la liste des 10 mots les plus fréquents (donc triés par ordre décroissant de fréquence) : `word_counts.takeOrdered(10, lambda i: -i[1])`.
- Le reste des actions dans la [documentation officielle ici](#)

RDD : Transformations & Actions (Résumé)

Transformations	$map(f : T \Rightarrow U) : RDD[T] \Rightarrow RDD[U]$ $filter(f : T \Rightarrow Bool) : RDD[T] \Rightarrow RDD[T]$ $flatMap(f : T \Rightarrow Seq[U]) : RDD[T] \Rightarrow RDD[U]$ $sample(fraction : Float) : RDD[T] \Rightarrow RDD[T]$ (Deterministic sampling) $groupByKey() : RDD[(K, V)] \Rightarrow RDD[(K, Seq[V])]$ $reduceByKey(f : (V, V) \Rightarrow V) : RDD[(K, V)] \Rightarrow RDD[(K, V)]$ $union() : (RDD[T], RDD[T]) \Rightarrow RDD[T]$ $join() : (RDD[(K, V)], RDD[(K, W)]) \Rightarrow RDD[(K, (V, W))]$ $cogroup() : (RDD[(K, V)], RDD[(K, W)]) \Rightarrow RDD[(K, (Seq[V], Seq[W]))]$ $crossProduct() : (RDD[T], RDD[U]) \Rightarrow RDD[(T, U)]$ $mapValues(f : V \Rightarrow W) : RDD[(K, V)] \Rightarrow RDD[(K, W)]$ (Preserves partitioning) $sort(c : Comparator[K]) : RDD[(K, V)] \Rightarrow RDD[(K, V)]$ $partitionBy(p : Partitioner[K]) : RDD[(K, V)] \Rightarrow RDD[(K, V)]$
Actions	$count() : RDD[T] \Rightarrow Long$ $collect() : RDD[T] \Rightarrow Seq[T]$ $reduce(f : (T, T) \Rightarrow T) : RDD[T] \Rightarrow T$ $lookup(k : K) : RDD[(K, V)] \Rightarrow Seq[V]$ (On hash/range partitioned RDDs) $save(path : String) : \text{Outputs RDD to a storage system, e.g., HDFS}$

- **Remarque :** les transformations ne sont pas évaluées immédiatement. Elles sont évaluées de manière paresseuse ("*lazy evaluation*"), c'est à dire uniquement lorsqu'on effectue une action.

Spark Job Scheduling



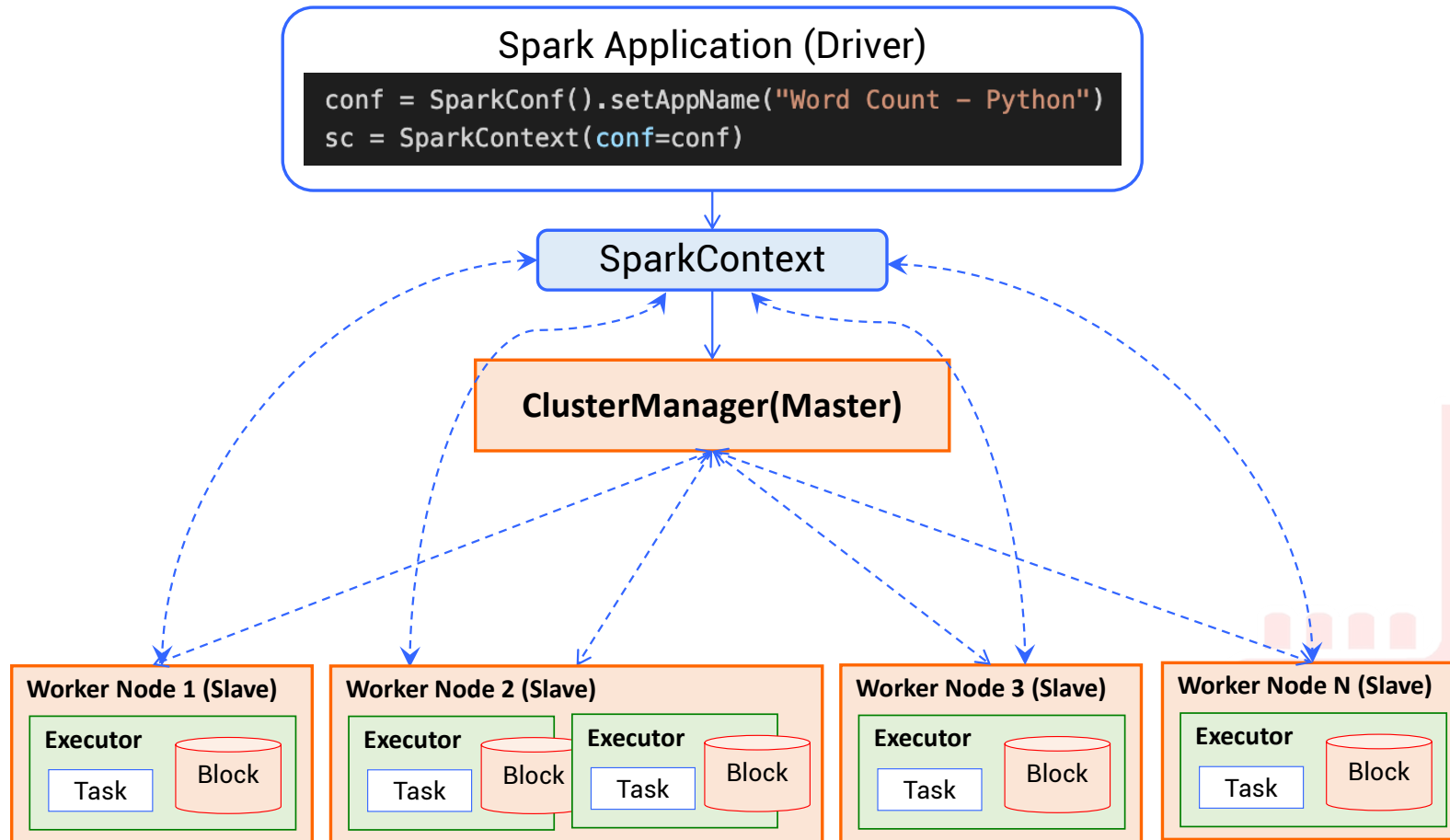
DAG Scheduler : Plan d'exécution

- Le plan d'exécution indique comment Spark exécute un programme ou une application
- Exemple

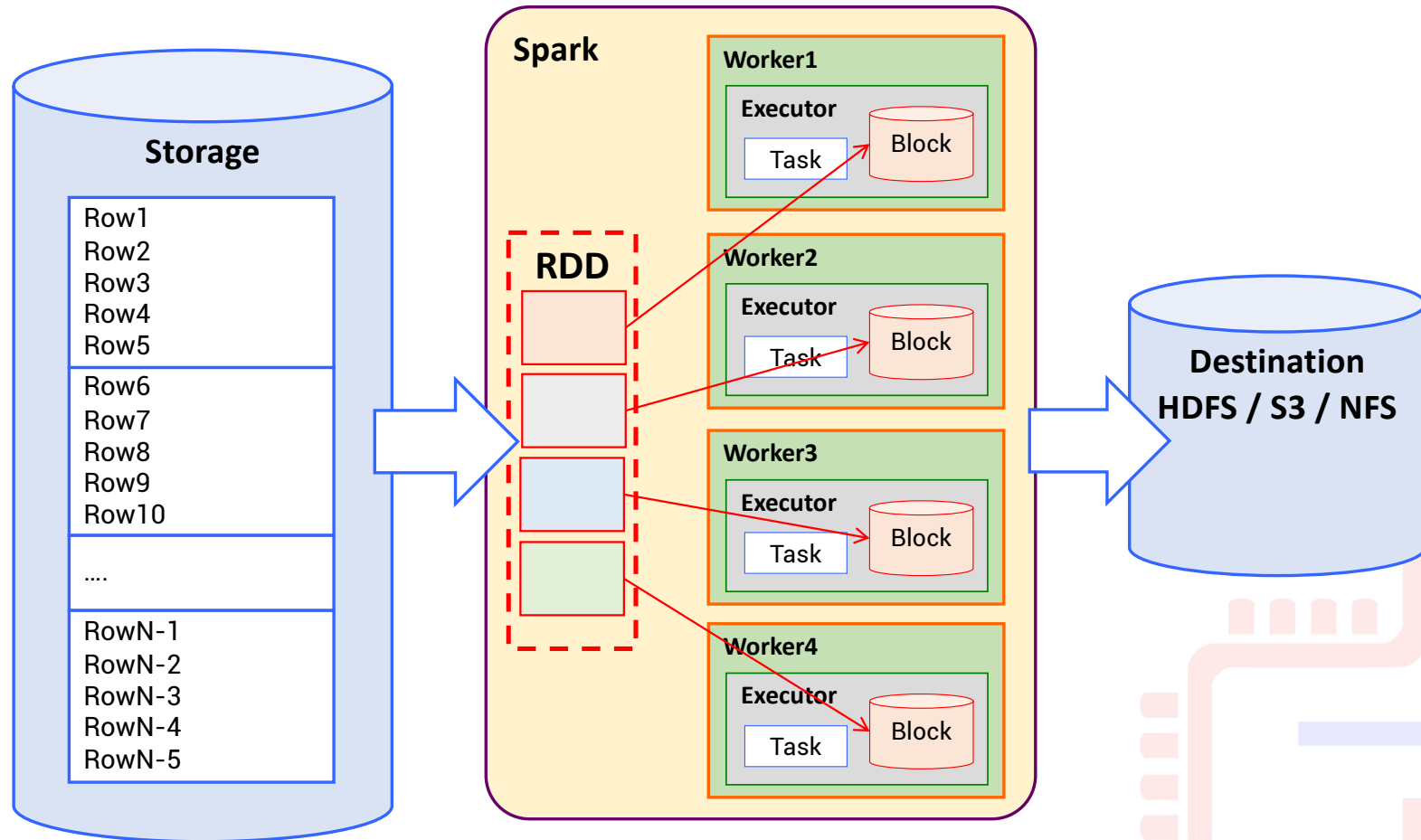
```
counts = sc.textFile("/path/to/input/") \
    .flatMap(lambda line: line.split(" ")) \
    .map(lambda word: (word, 1)) \
    .reduceByKey(lambda a, b: a + b)
counts.saveAsTextFile("/path/to/output/")
```

- Les opérations effectuées dans ce programme
 - Tâche 1: chargement d'une entrée dans un RDD
 - Tâche 2: pré-traitement
 - Tâche 3: map
 - Tâche 4: reduce
 - Tâche 5: Enregistrement
- **Remarque** un programme est exécuté sur un Cluster (Master avec plusieurs Worker)

Batch Processing

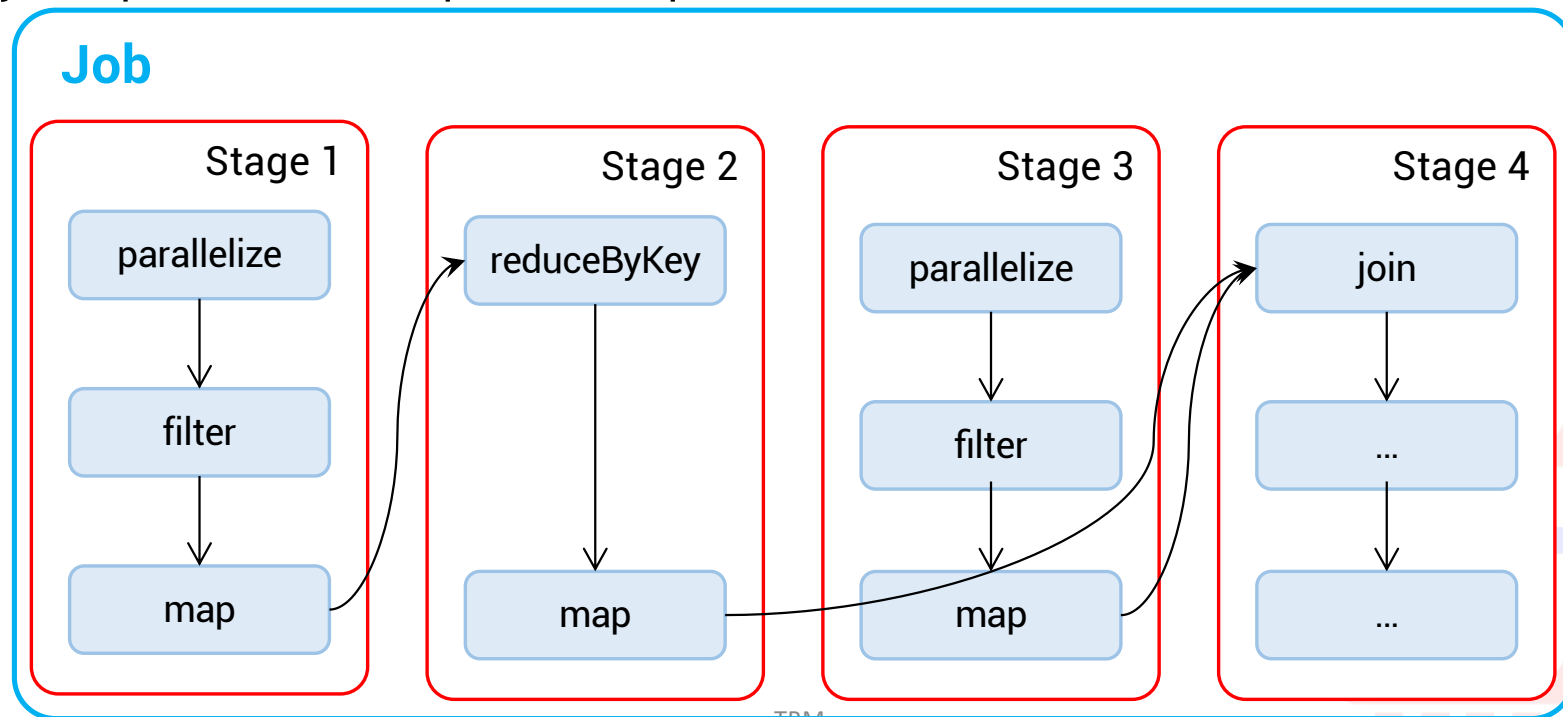


RDD : Notion de partition



Plan d'exécution

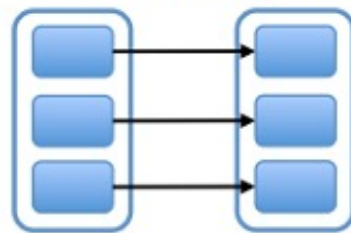
- Un **Job Spark** est composé d'une succession d'**étapes (Stage)**
 - La progression d'un job peut donc être mesurée au nombre d'étapes qui ont été réalisées.
- Un job Spark est créé pour chaque **action** réalisée sur un RDD



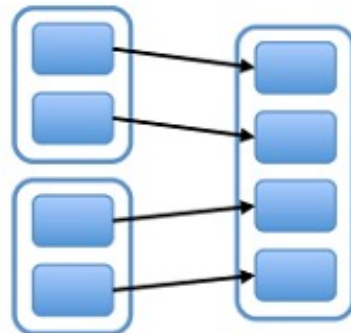
Narrow and Wide Transformations

- Examples

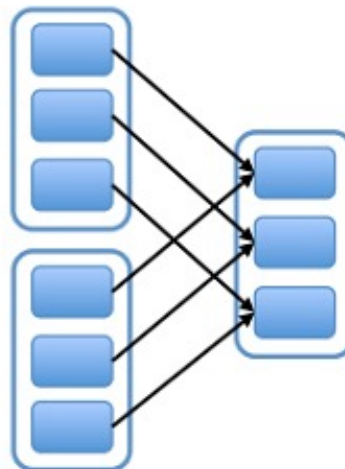
Narrow Dependencies:



map, filter

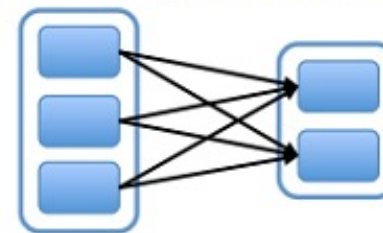


union

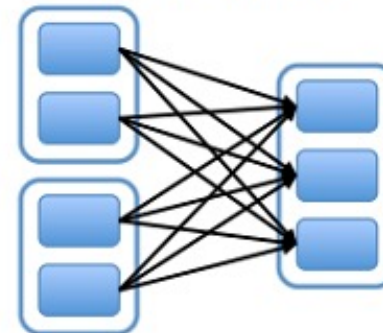


join with inputs
co-partitioned

Wide Dependencies:



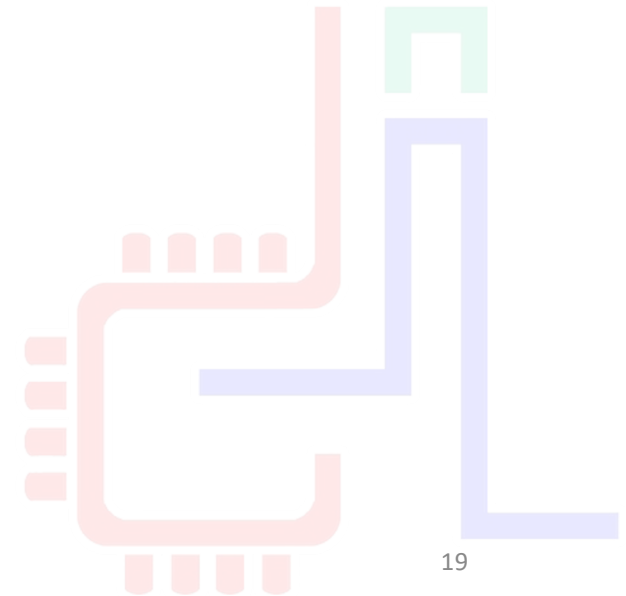
groupByKey



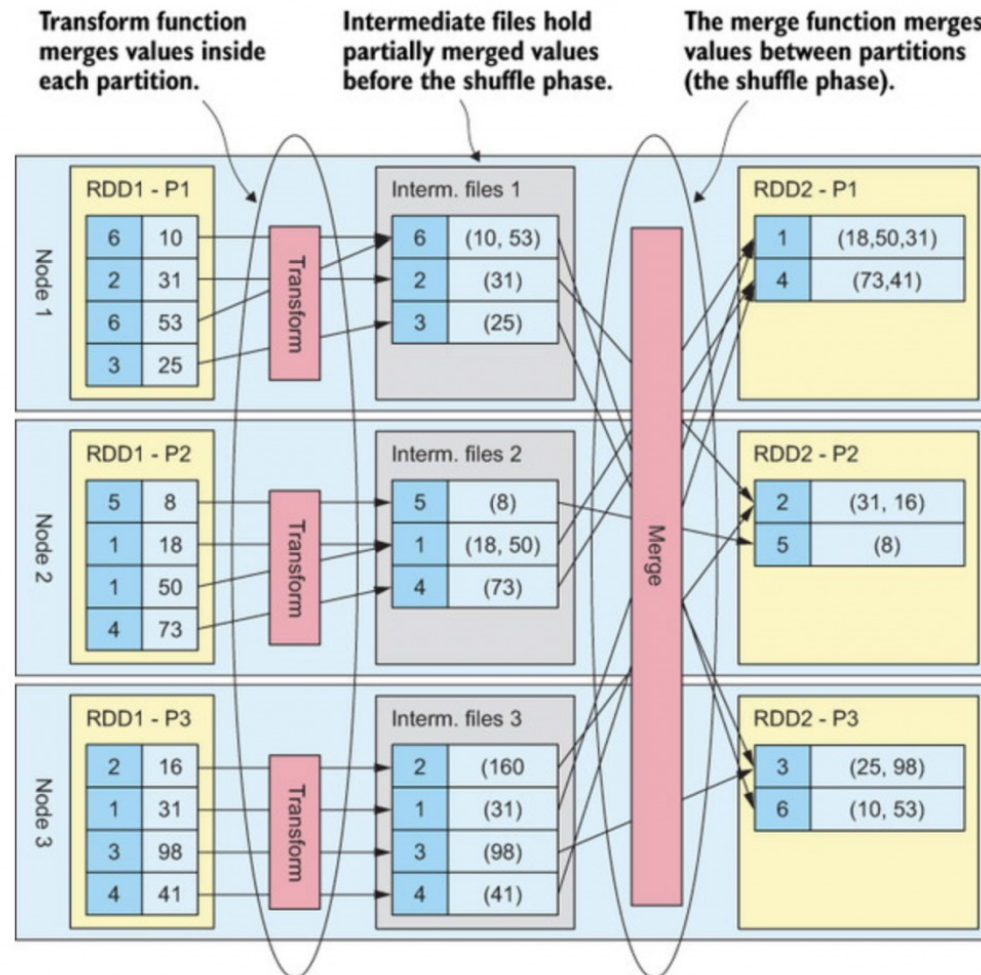
join with inputs not
co-partitioned

Transformation and Suffuling

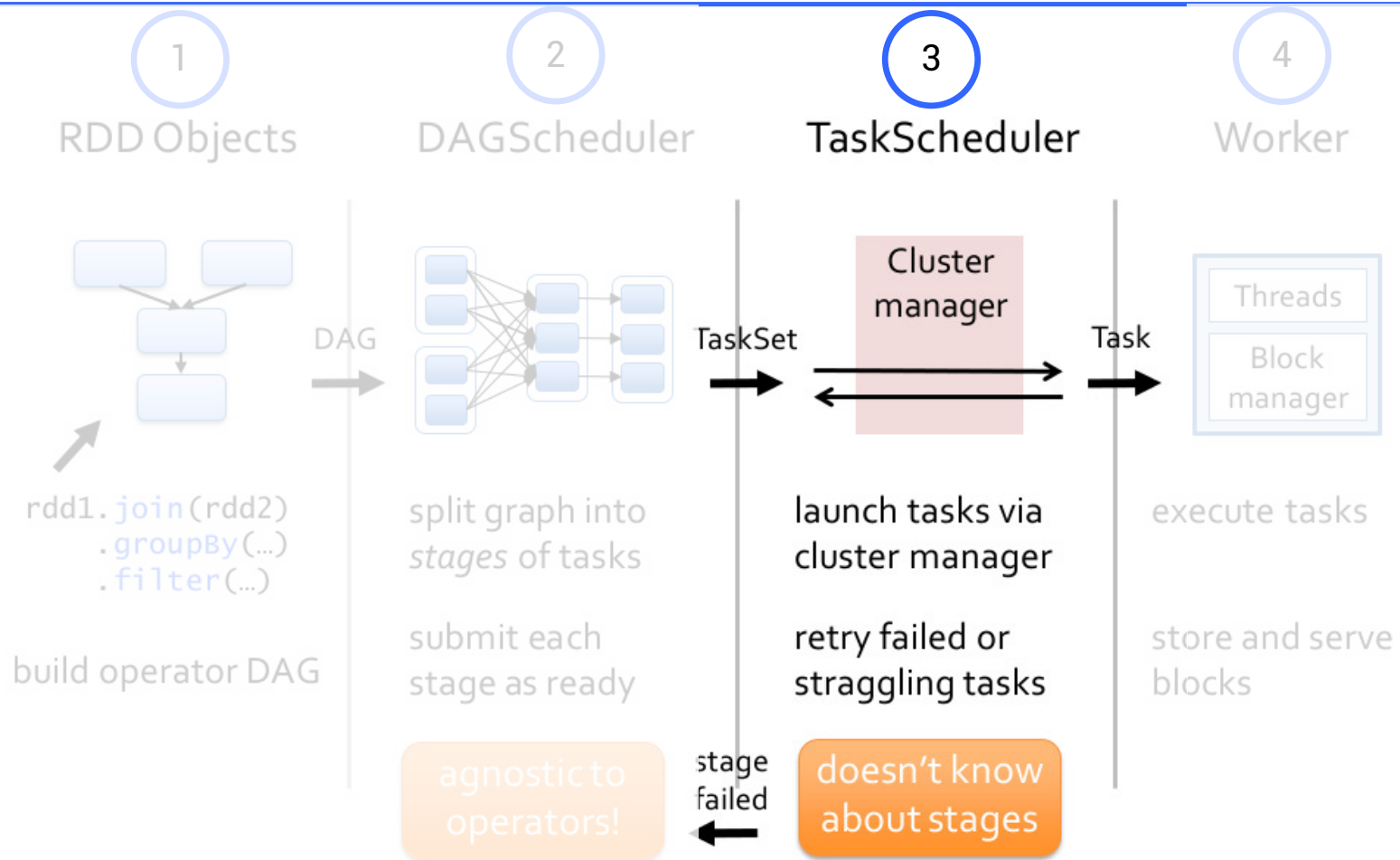
- Transformations causant un **Shuffle** :
 - Transformation Pair RDD (RDD (clé, valeur)) qui peuvent changer le Partitioner RDD:
 - `aggregateByKey`
 - `foldByKey`
 - `reduceByKey`
 - `groupByKey`
 - `join` et `leftOuterJoin`, `rightOuterJoin`, `fullOuterJoin`,
 - `subtractByKey`
 - RDD transformations:
 - `subtract`
 - `intersection`
 - `groupWith`
 - `sortByKey` déclenche toujours un shuffle
 - `partitionBy` or `coalesce` avec `shuffle=true`



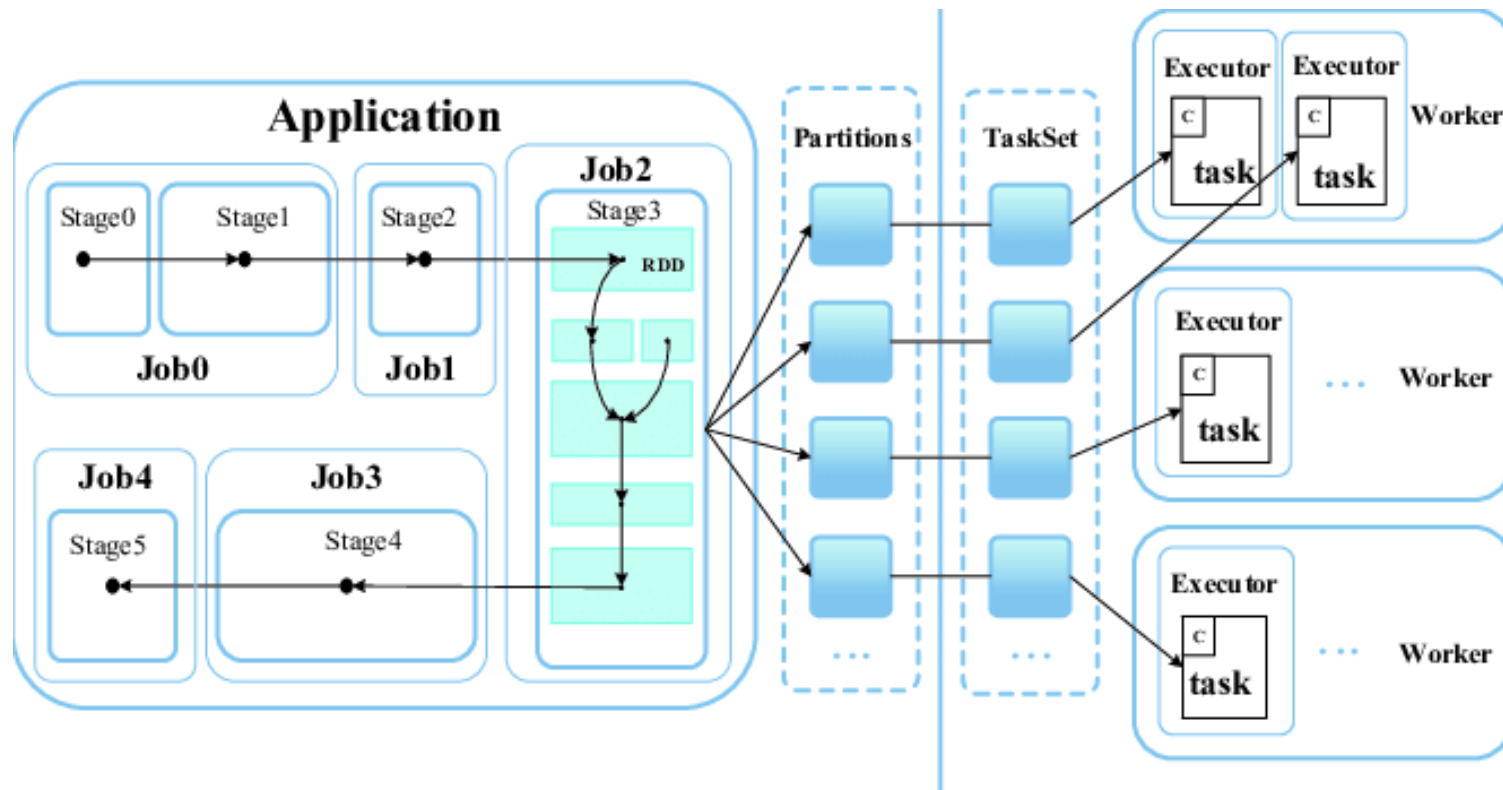
Exemple : Suffle durant aggregateByKey



Spark Job Scheduling



Taskset



Résumé

- Un **job** Spark correspond à une action sur un RDD et est composé de plusieurs **étapes** séparées par des **shuffles**.
- Chaque **étape** est composée de **tâches**.
- Chaque **tâche** s'exécute sur une **partition** différente des données.
- Les partitions sont créées par les **Resilient Distributed Datasets (RDD)**.
- Les **partitions** et leurs **tâches** correspondantes sont réparties sur les différents **executors**.



VB avec DataFrame

- DataFrame est un RDD **structuré** (assimilables à des tables relationnelles)

```
exBroadcast_df.py > ...
1  import pyspark
2
3  from pyspark.sql import SparkSession
4
5  spark = SparkSession.builder.appName('BroadCast Variables').getOrCreate()
6
7  states = {"TN":"Tunis", "BZ":"Bizerte", "KF":"Kef"}
8  broadcastStates = spark.sparkContext.broadcast(states)
9
10 data = [("Foulen","Ben Foulen","TUN","TN"),
11         ("Filtén","Ben Filtén","TUN","KF"),
12         ("Filténa","Bint Flen","TUN","BZ"),
13         ("Flen","Ben Mohamed","TUN","TN")
14     ]
15
16 columns = ["firstname","lastname","country","state"]
17 df = spark.createDataFrame(data = data, schema = columns)
18 df.printSchema()
19 df.show(truncate=False)
20
21 def state_convert(code):
22     return broadcastStates.value[code]
23 #Affichage sous forme Data Frame
24 result = df.rdd.map(lambda x: (x[0],x[1],x[2],state_convert(x[3]))).toDF()
25 result.show(truncate=False)
26 #Conversion DF ==> RDD et Affichage sous forme RDD
27 result = result.rdd.map(tuple).collect()
28 print(result)
```

```
root
|-- firstname: string (nullable = true)
|-- lastname: string (nullable = true)
|-- country: string (nullable = true)
|-- state: string (nullable = true)
```

firstname	lastname	country	state
Foulen	Ben Foulen	TUN	TN
Filtén	Ben Filtén	TUN	KF
Filténa	Bint Flen	TUN	BZ
Flen	Ben Mohamed	TUN	TN

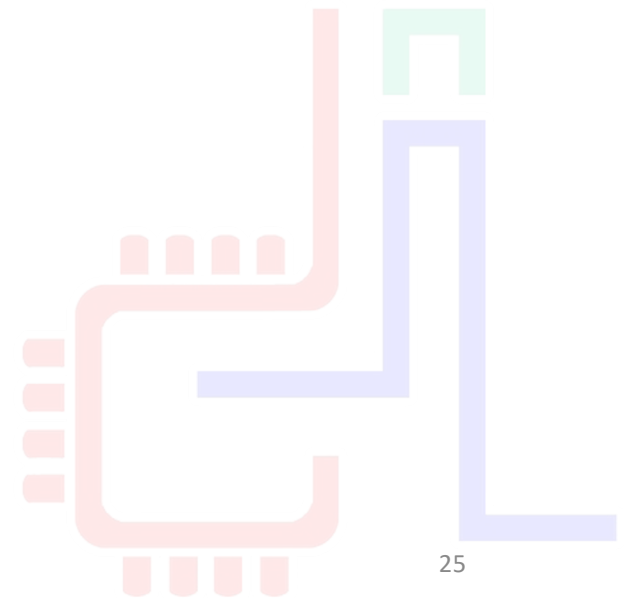
_1	_2	_3	_4
Foulen	Ben Foulen	TUN	Tunis
Filtén	Ben Filtén	TUN	Kef
Filténa	Bint Flen	TUN	Bizerte
Flen	Ben Mohamed	TUN	Tunis

```
[('Foulen', 'Ben Foulen', 'TUN', 'Tunis'), ('Filtén', 'Ben Filtén', 'TUN', 'Kef'), ('Filténa', 'Bint Flen', 'TUN', 'Bizerte'), ('Flen', 'Ben Mohamed', 'TUN', 'Tunis')]
```

PARTITIONNEMENT DES RDD

2K20

TBM



25

Notion de partition

- Un RDD représente de la donnée "distribuée", immuable, pouvant être traitée de façon parallèle.
- Une **partition** représente **une partie de la donnée d'un RDD**.

- **Exemple**

```
>>> rdd = sc.parallelize(range(0,9))  
>>> print(rdd.getNumPartitions())  
4
```

- **Explication** : en mode local, le nombre de partitions par défaut d'un RDD est égal au nombre de cœurs que possède la machine
- **Remarque** : En mode cluster, ce RDD aurait eu un nombre de partitions égal au total des cores disponibles sur les executors du cluster.

- Les partitions ressembleront à

	Partition 1	Partition 3	Partition 3	Partition 4
RDD	[0,1]	[2,3,4]	[5,6]	[7,8,9]



le **nombre de partitions** d'un RDD = le **nombre de tasks** qui seront créées lors de l'exécution des traitements sur les données

Partition exemple

■ Exemple

```
[>>> rdd = sc.parallelize(range(0,9))  
[>>> rdd.map(lambda x: x*2).collect()  
[0, 2, 4, 6, 8, 10, 12, 14, 16]
```

- Ce code générera 4 *tasks* correspondantes à la fonction lambda (x2)

Executors

Show 20 entries

Search:

Executor ID	Address	Status	RDD Blocks	Storage Memory	Disk Used	Cores	Active Tasks	Failed Tasks	Complete Tasks	Total Tasks	Task Time (GC Time)	Input	Shuffle Read	Shuffle Write	Thread Dump
driver	192.168.1.20:53516	Active	0	3.5 KiB / 434.4 MiB	0.0 B	4	0	0	12	12	5 s (0.0 ms)	0.0 B	0.0 B	0.0 B	Thread Dump

Tasks (4)

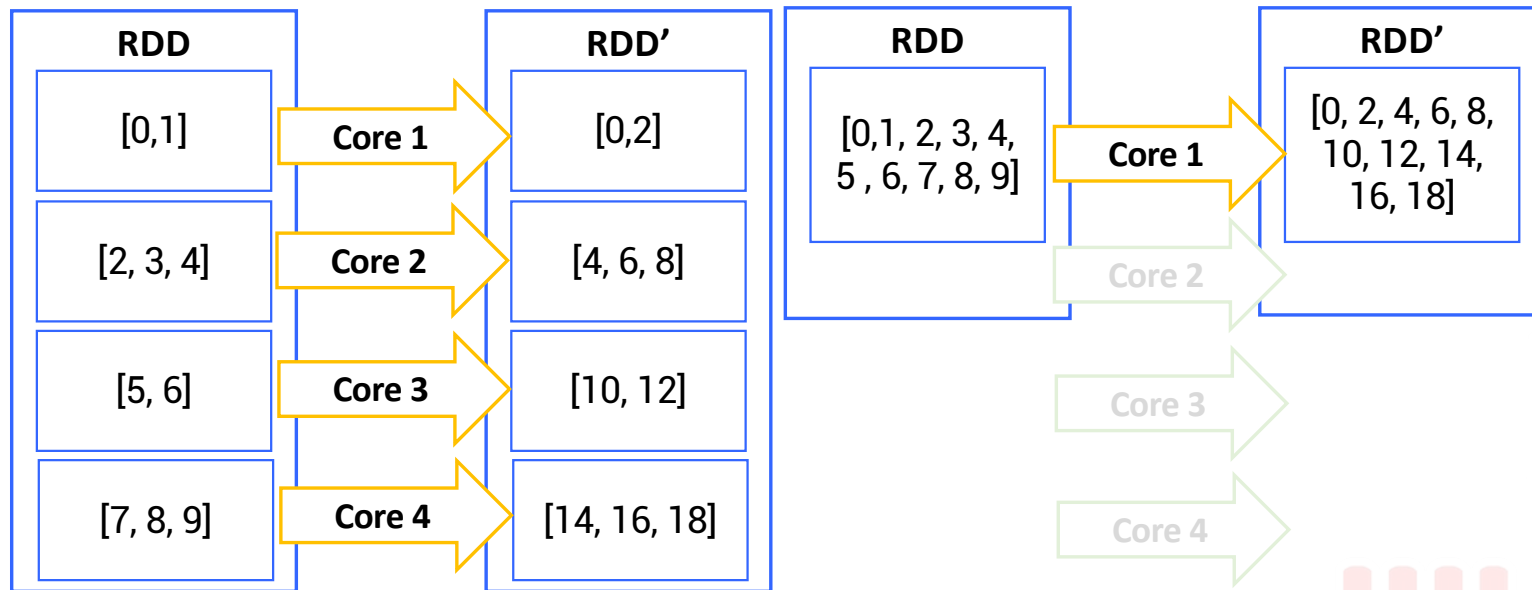
Show 20 entries

Search:

Index	Task ID	Attempt	Status	Locality level	Executor ID	Host	Logs	Launch Time	Duration	GC Time	Errors
0	4	0	SUCCESS	PROCESS_LOCAL	driver	192.168.1.20		2020-10-18 19:48:51	64.0 ms		
1	5	0	SUCCESS	PROCESS_LOCAL	driver	192.168.1.20		2020-10-18 19:48:51	68.0 ms		
2	6	0	SUCCESS	PROCESS_LOCAL	driver	192.168.1.20		2020-10-18 19:48:51	59.0 ms		
3	7	0	SUCCESS	PROCESS_LOCAL	driver	192.168.1.20		2020-10-18 19:48:51	70.0 ms		

Partition : Bonne pratique

- Comment fonctionnent les *tasks*?

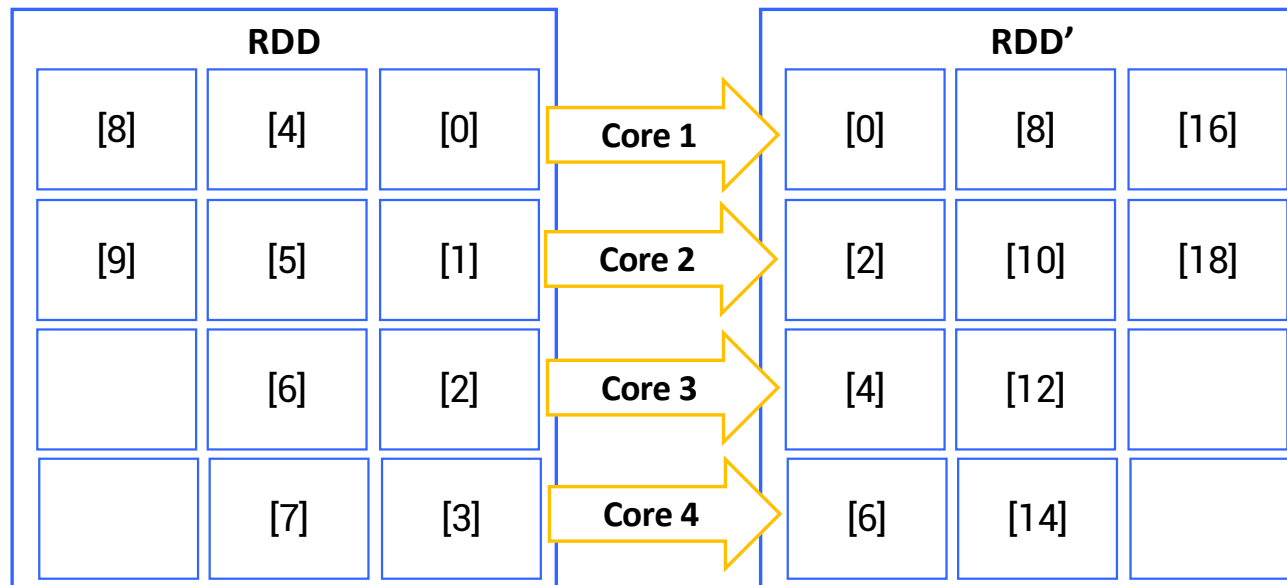


4 partitions et 4 cœurs, le traitement s'est correctement réparti

Avec 1 partition, utilisation d'un seul Core sur 4

Partition : Bonne pratique

- Et si on augmente nombre de partition

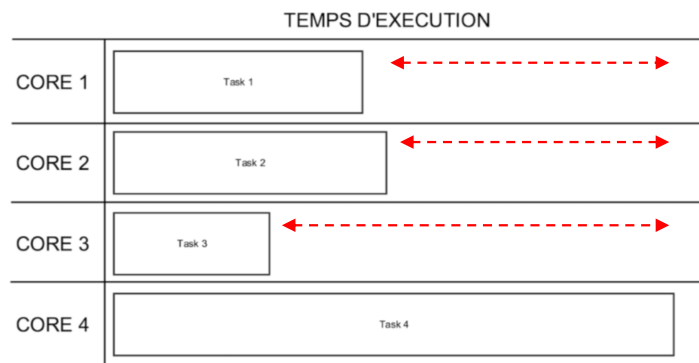


- Avec 12 partitions (> 4 nb cores), utilisation optimale de tous les cores.
 - Sachant qu'un core n'exécute qu'une task à la fois.
 - Aucun gain en parallélisme, mais augmentation des overhead (temps de gestion de ces tasks, sérialisation/désérialisation, etc.)

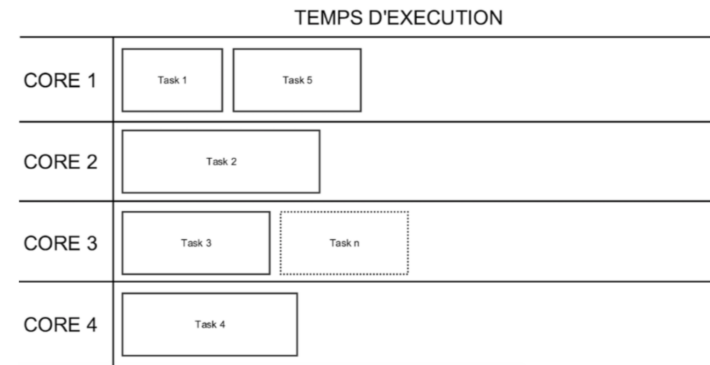
Partition : Bonne pratique

■ Remarque :

- Bien qu'un core n'exécute qu'une seule task à la fois, il est possible d'allouer plusieurs cores par task (multithreading).
 - Ex avec 6 cores et le paramètre `spark.task.cpus` à 2 ==> 3 tasks en parallèle au maximum.
- La documentation Spark recommande **3 partitions par core**.



Le temps de calcul n'est pas correctement réparti (raisons: différence de taille des partitions ou performances des cores, ...)



dès qu'un core termine un traitement, il se verra attribuer une autre task restée en attente

Partition : RangePartitioner

- Soit

```
[>>> rdd = sc.parallelize([(8, 'a'), (96, 'b'), (240, 'c'), (400, 'd'), (800, 'e')])
[>>> rdd_p = rdd.partitionBy(4)
[>>> rdd_p.glom().collect()
[[[8, 'a'), (96, 'b'), (240, 'c'), (400, 'd'), (800, 'e')], [], [], []]
```

- Le modulo du hash = 0 toute les données sans dans une même partition

- utilisation du *RangePartitioner* est plus judicieuse.

- il permet de regrouper des "ranges" de clés ensemble.

```
[>>> rdd = sc.parallelize([(8, 'a'), (96, 'b'), (240, 'c'), (400, 'd'), (800, 'e')])
[>>> df = rdd.toDF()
[>>> df.show()
+---+---+
| _1| _2|
+---+---+
|  8|  a|
| 96|  b|
|240|  c|
|400|  d|
|800|  e|
+---+---+

[>>> df.repartitionByRange(4, "_1")
DataFrame[_1: bigint, _2: string]
[>>> df.rdd.map(tuple).glom().collect()
[[[8, 'a')], [[96, 'b')], [[240, 'c')], [[400, 'd'), (800, 'e')]]
```